OFICINA POSTGRESQL — PIE IV

Criando um Banco de Dados do Zero

APOSTILA DO ALUNO — Material Autocontido

UFR — Projetos Integrados de Extensão | ETEC — Desenvolvimento de Sistemas

Bem-vindo à Oficina

Esta apostila foi criada para que você consiga acompanhar, entender e reproduzir cada etapa da oficina — inclusive depois, sozinho, sem precisar de internet ou de ninguém ao lado. Não é necessária nenhuma experiência anterior com banco de dados.

Ao final desta oficina você terá construído, do absoluto zero, um banco de dados PostgreSQL com usuários configurados, permissões por perfil e dados reais populados — pronto para ser explorado com SQL na próxima aula.

Pré-requisitos

- **PostgreSQL 17** instalado no Windows
- **psql** configurado no PATH do sistema (veja o passo a seguir)
- **DBeaver** instalado — será usado na próxima aula

CONFIGURANDO O PATH DO PSQL

Após instalar o PostgreSQL, o comando `psql` pode não funcionar no CMD. O motivo: o Windows não sabe onde o programa está instalado. Precisamos informá-lo.

1. Tecla Windows → buscar **Variáveis de ambiente** → abrir
2. Em **Variáveis do sistema**, localizar `Path` → clicar em **Editar**
3. Clicar em **Novo** e adicionar: `C:\Program Files\PostgreSQL\17\bin`
4. Confirmar com OK em todas as janelas abertas
5. **Fechar completamente o CMD e abrir um novo**

> ⚠ Este último passo é crítico. A janela do CMD aberta antes da alteração nunca vai reconhecer o novo caminho. Sempre abra uma janela nova.

VERIFICANDO O SERVIÇO POSTGRESQL

O PostgreSQL funciona como um serviço do Windows — um programa que roda em segundo plano. Ele precisa estar em execução para aceitar conexões.

```
sc query postgresql-x64-17
```

Se o estado exibido for **RUNNING**, está pronto para uso. Se for **STOPPED**:

```
net start postgresql-x64-17
```

> ⚠ Para iniciar ou parar o serviço é necessário abrir o CMD como **administrador**: botão direito no ícone do CMD → "Executar como administrador".

Conceitos Fundamentais — Entendendo o PostgreSQL

Antes de escrever qualquer comando, vamos entender como o PostgreSQL organiza os dados. Esses conceitos são a base de tudo que faremos a seguir.

A ANALOGIA DO PRÉDIO

Imagine o PostgreSQL como um **prédio corporativo**. Dentro desse prédio existe uma estrutura de organização muito clara:

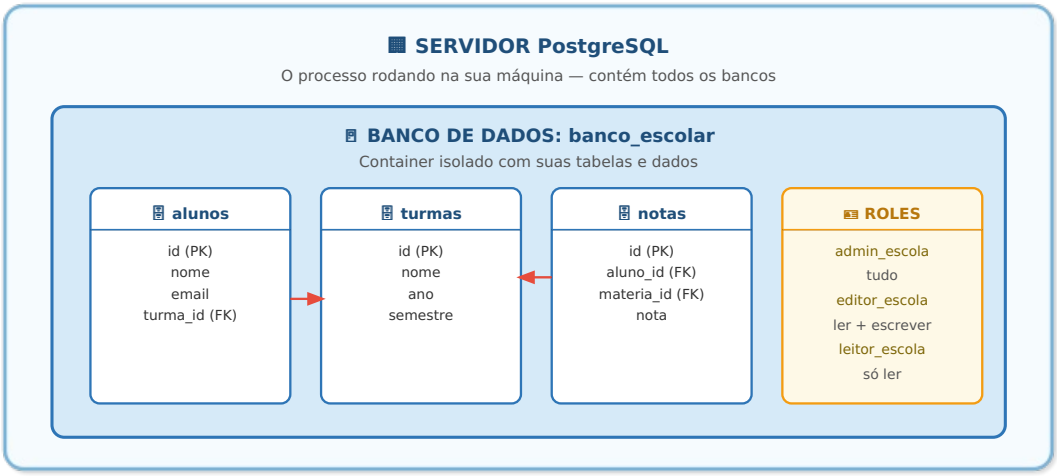


Figura 1 — Analogia visual da estrutura do PostgreSQL

- **Servidor (o prédio):** é o processo do PostgreSQL rodando na sua máquina. É a estrutura maior que contém tudo. Um único servidor pode hospedar dezenas de bancos de dados diferentes, completamente isolados uns dos outros.
- **Banco de dados (a sala):** cada banco é um espaço isolado dentro do servidor. O que está em um banco não interfere no outro. É como duas salas com portas fechadas — o que acontece em uma, a outra não sabe.
- **Tabela (o armário):** dentro de cada banco existem tabelas — são os "armários" onde os dados ficam organizados em linhas e colunas, como uma planilha.
- **Role (o crachá):** toda pessoa que acessa o PostgreSQL precisa de uma role — uma identidade com permissões definidas. É o crachá de acesso: define quem você é e o que você pode fazer dentro do prédio.
- **Schema (os corredores):** dentro de um banco, um schema é como um corredor que agrupa tabelas relacionadas. Nesta oficina usaremos sempre o schema padrão, chamado `public`.
- **GRANT / REVOKE (as regras de acesso):** comandos que concedem (`GRANT`) ou revogam (`REVOKE`) permissões específicas para cada role — como configurar o que cada crachá pode abrir.

> 💡 Resumo mental: servidor = prédio | banco = sala | tabela = armário | role = crachá | GRANT = regra do crachá

Chaves Primárias e Estrangeiras

Dois dos conceitos mais importantes em banco de dados relacional são as **chaves primárias** e as **chaves estrangeiras**. Elas são o que permite ligar tabelas entre si e garantir a integridade dos dados.

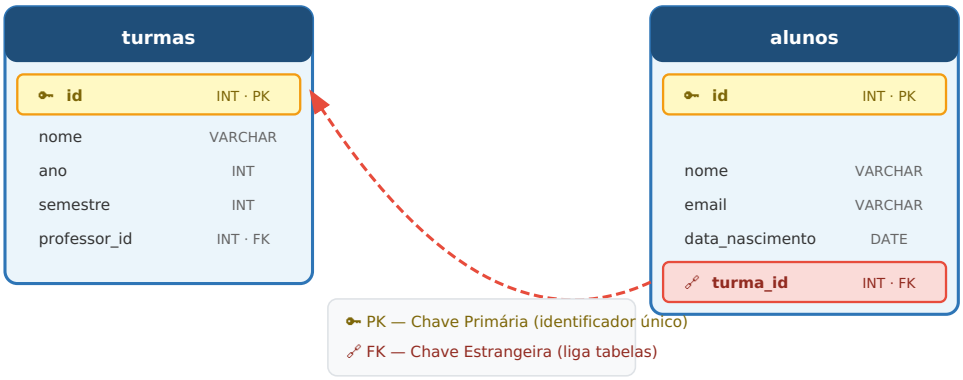


Figura 2 — Relação entre Chave Primária (PK) e Chave Estrangeira (FK)

CHAVE PRIMÁRIA (PRIMARY KEY)

> 🔑 A **Chave Primária** é o identificador único de cada linha em uma tabela. Nenhuma linha pode ter o mesmo valor de PK — ela garante que cada registro é único e pode ser encontrado.

No PostgreSQL, usamos o tipo `SERIAL PRIMARY KEY` — isso cria um número inteiro que se incrementa automaticamente (1, 2, 3...) a cada novo registro inserido. Você não precisa informar esse valor, o banco preenche sozinho.

Exemplo: na tabela `turmas`, o campo `id` é a PK. Se você inserir 4 turmas, elas terão IDs 1, 2, 3 e 4 automaticamente.

```
CREATE TABLE turmas (
    id   SERIAL PRIMARY KEY,    -- gerado automaticamente: 1, 2, 3...
    nome VARCHAR(50) NOT NULL
);
```

CHAVE ESTRANGEIRA (FOREIGN KEY)

> 🔗 A **Chave Estrangeira** é um campo em uma tabela que aponta para a Chave Primária de outra tabela. É o mecanismo que cria o **relacionamento** entre tabelas.

Quando dizemos que um aluno pertence a uma turma, fazemos isso com uma FK: o campo `turma_id` na tabela `alunos` guarda o `id` da turma correspondente na tabela `turmas`.

```
CREATE TABLE alunos (
    id       SERIAL PRIMARY KEY,
    nome     VARCHAR(100) NOT NULL,
    turma_id INT NOT NULL REFERENCES turmas(id)  -- FK: aponta para turmas.id
);
```

A palavra-chave `REFERENCES` diz ao PostgreSQL: "o valor deste campo deve existir na tabela referenciada". Se você tentar inserir um aluno com `turma_id = 99` e não existir uma turma com `id = 99`, o banco recusará o INSERT com erro. Isso é proteção de integridade.

💡 Regra prática: sempre crie primeiro a tabela que será referenciada (a que tem a PK), depois a que referencia (a que tem a FK). É por isso que criamos as dimensões antes dos fatos.

Tabelas Dimensão e Tabelas Fato

Em banco de dados, existe uma distinção importante entre dois tipos de tabelas: as de **dimensão** e as de **fato**. Entender essa diferença ajuda a organizar melhor os dados e a pensar em como as consultas serão feitas.

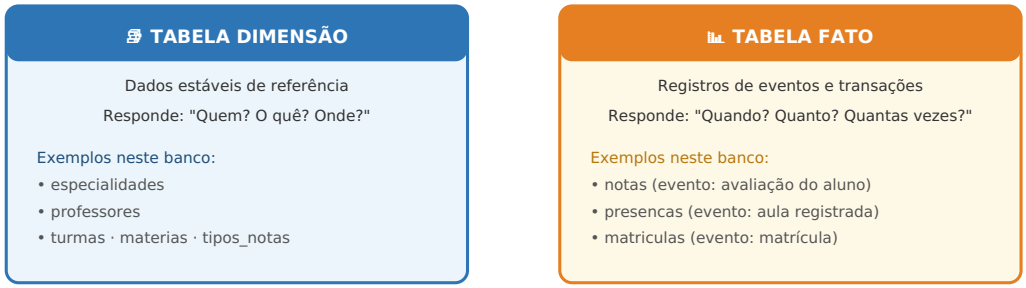


Figura 3 — Diferença entre Tabela Dimensão e Tabela Fato

TABELAS DIMENSÃO

🗃️ Uma **Tabela Dimensão** armazena dados de referência — informações que descrevem "quem", "o quê" ou "onde". Esses dados mudam pouco com o tempo e servem como base para outras tabelas.

No Banco Escolar, as dimensões são:

- **especialidades** — as áreas de conhecimento dos professores
- **professores** — quem leciona
- **turmas** — em qual grupo o aluno está
- **materias** — o que é ensinado
- **tipos_notas** — como as avaliações se classificam

TABELAS FATO

📊 Uma **Tabela Fato** armazena eventos e transações — registros do que aconteceu, quando aconteceu e em quais dimensões. São as tabelas que mais crescem com o tempo.

No Banco Escolar, as tabelas fato são:

- **notas** — o evento "um aluno recebeu uma nota em uma matéria"
- **presencas** — o evento "um aluno esteve ou não em uma aula"
- **matriculas** — o evento "um aluno se matriculou em uma turma"

Note que as tabelas fato quase sempre têm várias Chaves Estrangeiras — cada uma apontando para uma dimensão diferente. Isso é o que torna os dados relacionais: ao consultar uma nota, você consegue saber quem é o aluno, qual é a matéria e qual tipo de avaliação foi, tudo via FK.

Diagrama Completo do Banco Escolar

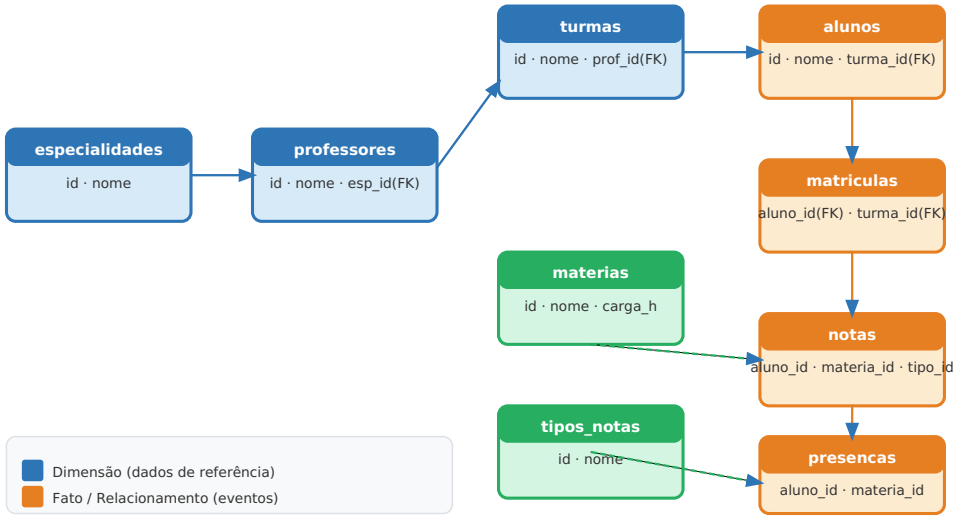


Figura 4 — Diagrama de relacionamentos do Banco Escolar (azul = dimensão, laranja = fato, verde = apoio)

Conectando ao PostgreSQL via CMD

```
psql -U postgres
```

O prompt mudará para `postgres=#` — isso indica que você está conectado ao servidor como superusuário. Explore o ambiente com estes comandos:

```
\l        -- lista todos os bancos de dados existentes
\du       -- lista todas as roles existentes
\conninfo -- mostra em qual banco e usuário você está
```

Por padrão, você verá apenas os bancos de sistema (`postgres` , `template0` , `template1`) e a role `postgres` . Agora vamos construir nossa própria estrutura.

Criando as Roles

Trabalharemos com 3 roles, cada uma com um nível diferente de acesso:

Role	Perfil	O que pode fazer
`admin_escola`	Superusuário	Controle total — criar, alterar, excluir qualquer coisa
`editor_escola`	Editor	Ler, inserir, atualizar e excluir dados nas tabelas
`leitor_escola`	Leitor	Apenas consultar (SELECT) — não pode modificar nada

SUPER ADMIN

```
CREATE ROLE admin_escola WITH
    LOGIN
    SUPERUSER
    PASSWORD 'Admin@2026';
```

`LOGIN` permite que a role se autentique diretamente no servidor, como um usuário. `SUPERUSER` concede poder irrestrito — é o administrador total do banco.

EDITOR

```
CREATE ROLE editor_escola WITH
    LOGIN
    PASSWORD 'Editor@2026';
```

Sem `SUPERUSER` . Os privilégios específicos (o que ela pode fazer em cada tabela) serão concedidos depois com `GRANT` . Criar a role aqui apenas define a identidade.

LEITOR

```
CREATE ROLE leitor_escola WITH
    LOGIN
    PASSWORD 'Leitor@2026';
```

```
\du   -- verifique as 4 roles criadas
```

✔ Você deve ver: postgres, admin_escola, editor_escola, leitor_escola

> ⚠ Senhas em texto plano como essas são apenas para fins didáticos. Em produção, use senhas fortes e ferramentas de gestão de segredos (vaults).

Criando o Banco de Dados

```
CREATE DATABASE banco_escolar
    OWNER admin_escola
    ENCODING 'UTF8';
```

`OWNER` define quem tem controle total sobre esse banco — usamos o `admin_escola` que acabamos de criar. `ENCODING 'UTF8'` garante suporte completo a acentos, cedilha e caracteres especiais.

Saia e reconecte como admin no novo banco:

```
\q
```

```
psql -U admin_escola -d banco_escolar
```

```
\conninfo   -- confirme: banco = banco_escolar, user = admin_escola
```

> ✔ Conexão confirmada no banco e usuário corretos

Criando as Tabelas

Com o banco criado, vamos montar a estrutura do Banco Escolar. Lembre da regra: **dimensões primeiro, fatos depois**.

DIMENSÕES (CRIAR PRIMEIRO)

```
CREATE TABLE especialidades (
    id   SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);

CREATE TABLE professores (
    id               SERIAL PRIMARY KEY,
    nome             VARCHAR(100) NOT NULL,
    email            VARCHAR(100) NOT NULL,
    especialidade_id INT NOT NULL REFERENCES especialidades(id)
);

CREATE TABLE turmas (
    id           SERIAL PRIMARY KEY,
    nome         VARCHAR(50)  NOT NULL,
    ano          INT          NOT NULL,
    semestre     INT          NOT NULL,
    professor_id INT          NOT NULL REFERENCES professores(id)
);

CREATE TABLE materias (
    id            SERIAL PRIMARY KEY,
    nome          VARCHAR(100) NOT NULL,
    carga_horaria INT          NOT NULL DEFAULT 64
```

```
);

CREATE TABLE tipos_notas (
    id   SERIAL PRIMARY KEY,
    nome VARCHAR(50) NOT NULL
);
```

FATOS E RELACIONAMENTOS (CRIAR DEPOIS)

```
CREATE TABLE alunos (
    id              SERIAL PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    email           VARCHAR(100) NOT NULL,
    data_nascimento DATE,
    turma_id        INT NOT NULL REFERENCES turmas(id)
);

CREATE TABLE matriculas (
    id             SERIAL PRIMARY KEY,
    aluno_id       INT  NOT NULL REFERENCES alunos(id),
    turma_id       INT  NOT NULL REFERENCES turmas(id),
    data_matricula DATE NOT NULL
);

CREATE TABLE notas (
    id         SERIAL PRIMARY KEY,
    aluno_id   INT   NOT NULL REFERENCES alunos(id),
    materia_id INT   NOT NULL REFERENCES materias(id),
    nota       FLOAT NOT NULL,
    tipo_id    INT   NOT NULL REFERENCES tipos_notas(id)
);

CREATE TABLE presencas (
    id          SERIAL PRIMARY KEY,
    aluno_id    INT     NOT NULL REFERENCES alunos(id),
    materia_id  INT     NOT NULL REFERENCES materias(id),
    data_aula   DATE    NOT NULL,
    presente    BOOLEAN NOT NULL,
    justificada BOOLEAN
);
```

```
\dt   -- confirme as 9 tabelas criadas
```

✔ 9 tabelas listadas: especialidades, professores, turmas, materias, tipos_notas, alunos, matriculas, notas, presencas

---

Aplicando Permissões (GRANT)

As roles existem, mas ainda não têm acesso ao banco. O `GRANT` concede permissões específicas por role.

EDITOR — LEITURA E ESCRITA COMPLETAS

```
GRANT CONNECT ON DATABASE banco_escolar TO editor_escola;
GRANT USAGE ON SCHEMA public TO editor_escola;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO editor_escola;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO editor_escola;
```

O que cada linha faz:

- `CONNECT` — permite se conectar ao banco
- `USAGE ON SCHEMA` — permite enxergar o schema public e suas tabelas
- `SELECT, INSERT, UPDATE, DELETE` — as quatro operações de dados (CRUD)
- `USAGE, SELECT ON SEQUENCES` — necessário para usar o SERIAL (auto-incremento)

LEITOR — SOMENTE LEITURA

```
GRANT CONNECT ON DATABASE banco_escolar TO leitor_escola;
GRANT USAGE ON SCHEMA public TO leitor_escola;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO leitor_escola;
```

TESTANDO NA PRÁTICA

```
\q
psql -U leitor_escola -d banco_escolar
```

```
INSERT INTO especialidades (nome) VALUES ('Teste');
```

> ✔ Esperado: ERROR: permission denied for table especialidades — isso é correto!

Esse erro comprova que as permissões estão funcionando. Reconecte como `editor_escola` e repita o mesmo INSERT — desta vez deve funcionar.

Populando as Tabelas

```
\q
psql -U admin_escola -d banco_escolar
```

Execute o script completo em `dados/04_inserts.sql` do repositório. Ele contém todos os dados do Banco Escolar:

Tabela	Registros	O que representa
especialidades	5	Áreas dos professores
professores	8	Docentes da escola
turmas	4	Semestres 2024/1 a 2025/2
materias	6	Disciplinas com 64h cada
tipos_notas	3	Prova 1, Prova 2, Trabalho
alunos	20	5 alunos por turma
matriculas	60	Vínculos aluno-turma
notas	240	3 notas × 4 matérias × 20 alunos
presencas	960	16 aulas × 3 matérias × 20 alunos

```
SELECT COUNT(*) FROM alunos;      -- esperado: 20
SELECT COUNT(*) FROM presencas;   -- esperado: 960
```

> ✔ Se os contadores batem, o banco está completo e pronto para a próxima aula.

Resumo do que construímos

- ✔ Serviço PostgreSQL verificado e rodando
- ✔ 3 roles criadas com diferentes níveis de acesso
- ✔ Banco de dados `banco_escolar` criado
- ✔ 9 tabelas com chaves primárias e estrangeiras
- ✔ Permissões testadas e comprovadas na prática
- ✔ 960 registros de dados populados

Próxima aula — Oficina de SQL (PIE V)

Você vai conectar ao `banco_escolar` pelo **DBeaver** e explorar os dados com SQL:

- `SELECT`, `WHERE`, `ORDER BY`, `LIMIT`
- `GROUP BY`, `HAVING` e funções de agregação
- `INNER JOIN`, `LEFT JOIN`, `FULL OUTER JOIN`
- CTEs com `WITH` para consultas avançadas

Repositório: **github.com/ocarlossoueu/oficinasql**

Cheat Sheet — Comandos psql

Comando	Função
`\l`	Listar bancos
`\du`	Listar roles
`\dt`	Listar tabelas
`\conninfo`	Conexão atual
`\c nome_banco`	Trocar de banco
`\q`	Sair do psql
`\?`	Ajuda dos comandos
`\h COMANDO`	Ajuda de sintaxe SQL